Nombre: Jaime Sebastian Esparza Leyva Alan Joel Sánchez García Jorge Alejandro Jiménez González	**Matrícula**: 2097304 2082464 2097849
Nombre del curso: Componente de lógica de negocios	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: NA	**Actividad**: Resumen
Fecha: 15/10/2026	
Bibliografia (Formato APA)	

ENTREGABLE 06 · CÓDIGO BASE

Sazón Urbano · Arquitectura de software

Este entregable presenta una estructura base del repositorio y ejemplos mínimos de código para representar la arquitectura por capas propuesta para Sazón Urbano.

1. Estructura de carpetas del repositorio

```
sazon-urbano-architecture/
├── docs/
│   ├── informe_arquitectura.pdf
│   ├── trazabilidad.pdf
│   └── bitacora_ia.docx
├── diagrams/
│   ├── diagrama_dominio_clases.drawio
│   └── diagrama_secuencia.drawio
├── src/
│   ├── Domain/
│   │   ├── Aggregates/
│   │   │   ├── PedidoAggregate/
│   │   │   │   ├── Pedido.cs
│   │   │   │   ├── LineaPedido.cs
│   │   │   │   ├── EstadoPedido.cs
│   │   │   │   └── RegistroDeCambio.cs
│   │   │   ├── ProductoAggregate/
│   │   │   │   ├── Producto.cs
│   │   │   │   └── RequerimientoIngrediente.cs
│   │   │   └── IngredienteAggregate/
│   │   │       ├── Ingrediente.cs
│   │   │       └── MovimientoInventario.cs
│   │   ├── Services/
│   │   │   ├── DisponibilidadProductosService.cs
│   │   │   └── ConsumoDeIngredientesService.cs
│   │   ├── Repositories/
│   │   │   ├── IPedidoRepository.cs
│   │   │   └── IIngredienteRepository.cs
│   ├── Application/
│   │   ├── Services/ServicioDePedidos.cs
│   │   ├── Commands/
│   │   │   ├── ConfirmarPedidoCommand.cs
│   │   │   └── CancelarPedidoCommand.cs
│   │   ├── Queries/ConsultarOrdenesDeEstacionQuery.cs
│   │   └── Common/IUnitOfWork.cs
│   └── Infrastructure/
│       ├── Persistence/
│       │   ├── PedidoRepositorySql.cs
│       │   └── DbUnitOfWork.cs
│       └── Security/AutorizadorService.cs
└── README.md
```

2. Código mínimo demostrativo de la arquitectura

A. Modelo de dominio: Pedido.cs

Representa el pedido, controla sus cambios de estado y registra las acciones realizadas.

```
namespace SazonUrbano.Domain.Aggregates.PedidoAggregate
{
    public class Pedido
    {
        public Guid Id { get; private set; }
        public string CanalVenta { get; private set; }
        public EstadoPedido Estado { get; private set; }
        private readonly List<LineaPedido> _lineas = new();
```

```
        private readonly List<RegistroDeCambio> _historial = new();

        public IReadOnlyCollection<LineaPedido> Lineas => _lineas.AsReadOnly();
        public IReadOnlyCollection<RegistroDeCambio> Historial => _historial.AsReadOnly();

        public Pedido(Guid id, string canal, string usuario)
        {
            if (string.IsNullOrWhiteSpace(canal))
                throw new ArgumentException("El canal de venta es obligatorio.");

            Id = id;
            CanalVenta = canal;
            Estado = EstadoPedido.EnRegistro;
            RegistrarAuditoria("Creación de Pedido", usuario);
        }

        public void AgregarLinea(Guid productoId, int cantidad, string usuario)
        {
            if (Estado != EstadoPedido.EnRegistro)
                throw new InvalidOperationException("No se pueden agregar productos en el estado actual.");

            _lineas.Add(new LineaPedido(productoId, cantidad));
            RegistrarAuditoria($"Línea agregada: Producto {productoId}", usuario);
        }

        public void Confirmar(string usuario)
        {
            if (!_lineas.Any())
                throw new InvalidOperationException("No se puede confirmar un pedido sin productos.");

            Estado = EstadoPedido.Confirmado;
            RegistrarAuditoria("Pedido Confirmado para Cocina e Inventario", usuario);
        }

        public void Cancelar(string motivo, string usuario)
        {
            if (string.IsNullOrWhiteSpace(motivo))
                throw new ArgumentException("Debe indicar el motivo de la cancelación.");

            if (Estado == EstadoPedido.Listo || Estado == EstadoPedido.Cerrado)
                throw new InvalidOperationException("Este pedido ya no puede cancelarse.");

            Estado = EstadoPedido.Cancelado;
            RegistrarAuditoria($"Pedido Cancelado. Motivo: {motivo}", usuario);
        }

        private void RegistrarAuditoria(string accion, string usuario)
        {
            _historial.Add(new RegistroDeCambio(accion, usuario, DateTime.UtcNow));
        }
    }
}
```

B. Servicio de aplicación: ServicioDePedidos.cs

Coordina la confirmación del pedido, el descuento de inventario y la persistencia dentro de una transacción.

```
namespace SazonUrbano.Application.Services
{
    public class ServicioDePedidos
    {
        private readonly IPedidoRepository _pedidoRepo;
        private readonly IIngredienteRepository _ingredienteRepo;
        private readonly ConsumoDeIngredientesService _consumoService;
        private readonly IUnitOfWork _unitOfWork;

        public ServicioDePedidos(
```

```
            IPedidoRepository pedidoRepo,
            IIngredienteRepository ingredienteRepo,
            ConsumoDeIngredientesService consumoService,
            IUnitOfWork unitOfWork)
        {
            _pedidoRepo = pedidoRepo;
            _ingredienteRepo = ingredienteRepo;
            _consumoService = consumoService;
            _unitOfWork = unitOfWork;
        }

        public async Task ConfirmarPedidoAsync(Guid pedidoId, string usuarioId)
        {
            await _unitOfWork.BeginTransactionAsync();
            try
            {
                var pedido = await _pedidoRepo.ObtenerPorIdAsync(pedidoId);
                if (pedido == null) throw new KeyNotFoundException("Pedido no encontrado.");

                // 1. Invocación de reglas del dominio
                pedido.Confirmar(usuarioId);

                // 2. Descuento transaccional de inventario (ADR-03)
                await _consumoService.DescontarInsumosDePedidoAsync(pedido, _ingredienteRepo);

                // 3. Persistencia atómica
                await _pedidoRepo.GuardarAsync(pedido);
                await _unitOfWork.CommitAsync();
            }
            catch
            {
                await _unitOfWork.RollbackAsync();
                throw;
            }
        }
    }
}
```

C. Puerto de unidad de trabajo: IUnitOfWork.cs

Define las operaciones necesarias para iniciar, confirmar o revertir una transacción.

```
namespace SazonUrbano.Application.Common
{
    public interface IUnitOfWork
    {
        Task BeginTransactionAsync();
        Task CommitAsync();
        Task RollbackAsync();
    }
}
```